

Group Name: Cryptix

Student Name (with initials): Samarakoon S.M.S.G.

Student ID: 230562E

Module: Computer Security — Semester 5

Submission: Submission 1 — Individual Design Document

My slice of the system: Accessible Client — all authentication screens (register, login, enrol, verify, recovery); ARIA semantics, focus order, keyboard-only operation, Web Speech API prompts, audio cues, and high-contrast/large-text mode.

1. Introduction

This project is a multi-factor login system for people who are blind or have low vision, so it has to work from start to finish with just a screen reader, a keyboard, and audio, without needing to see the screen at all. As a group we split the system into five parts: three login factors (password/PIN, a one-time code, and a security key), a part that manages the whole login process and keeps it secure, and the part that shows everything to the user. That last part is mine.

My job wasn't to build new cryptography — the other four members cover that. My job was making sure the screens themselves don't ruin all that security work by being impossible to use. A QR-code-only way to set up a one-time code, or a CAPTCHA that only works if you can see it, would make the whole system pointless for the people it's meant to serve, no matter how solid the back end is. This document covers what I assumed, the diagrams I used to plan things out, the decisions I made and why, and a few places where accessibility and security disagreed with each other.

For context on why this matters: the WHO estimates that 2.2 billion people worldwide have some degree of vision impairment, 191 million of them blind [1]. Login is often one of the first places these users hit a wall online, which is why WCAG 2.2 has a rule specifically about it, SC 3.3.8 *Accessible Authentication (Minimum)* [4]. I used that as my main guide from the start, not something I added on at the end.

2. Assumptions

A few assumptions shaped everything else, so I want to list them clearly instead of leaving them hidden in the design.

- **Who the user is.** I assumed someone already comfortable with a screen reader and a keyboard — not a first-time assistive-tech user, and not someone who is both blind and deaf, since that would need Braille-display support I had no way of building or testing. That's a real limitation on who this design currently works for (see §8).
 - **Which screen readers.** I designed and tested against NVDA and VoiceOver, and treated JAWS as a nice-to-have rather than something I had to support. This isn't a random pick: WebAIM's most recent Screen Reader User Survey found NVDA is now the most-used screen reader overall — 65.6% of respondents, just ahead of JAWS at 60.5% — and VoiceOver is by far the most used on mobile at 70.6% [2]. NVDA is also free, which matters for a student project with no budget.
 - **Platform.** This is a web app that runs in a normal modern browser, not a native app. That's what the group decided to build, and it means screen readers, WebAuthn, and the Web Speech API all work without needing anything extra installed.
 - **The Web Speech API is optional, not something the design depends on.** Browser support for the speech part of that API isn't the same everywhere, and it's still a draft standard rather than a finished one [7], so I never let it be the only way information reaches the user — see decision 3 below.
 - **Recovery.** I'm assuming a locked-out user can still read a recovery email through their own screen reader on some device. The recovery flow itself belongs to another member, so it's outside my control, but it has to work for the whole system to actually work.
 - **Network.** I assumed a reasonably stable internet connection. This isn't built for offline use or a bad connection.
-

3. Requirements

3.1 Who I designed for

I used two personas to help me figure out what to build, which is a common way to approach this kind of design work before jumping into a build [12]:

- **Persona A** — total vision loss, expert screen-reader user. Navigates by headings and landmarks, never touches a mouse, and gets thrown off badly by an unexpected focus jump or a control with no label.
- **Persona B** — low vision, partially sighted. Uses screen magnification or browser zoom, needs strong colour contrast and larger text, and may still use a mouse or touch alongside the keyboard.

3.2 What has to be true (functional)

- Every screen — register, login step 1, OTP step, passkey step, enrolment, recovery — has to be fully usable with just a keyboard and a screen reader, monitor off.
- Anything that changes on the screen (a code accepted, moving to the next factor, getting locked out) has to be announced to assistive tech without a full page reload.
- The TOTP secret needs a non-QR way to get it — spoken or copyable text — since a QR code is a dead end for someone who can't use a camera scanner.
- A high-contrast/large-text mode has to be reachable from any screen, and the setting has to stick once turned on.

3.3 Mapping to WCAG 2.2

I matched each accessibility requirement to a specific WCAG 2.2 rule [3], so I had something concrete to check against instead of just a vague idea of "make it accessible":

Requirement	WCAG 2.2 criterion
Keyboard operable, no mouse-only control	2.1.1 Keyboard
No step traps focus / focus is always visible	2.4.7 Focus Visible, 2.4.11 Focus Not Obscured
Tab order matches visual/logical order	2.4.3 Focus Order
Generous, extendable timeouts on OTP/passkey steps	2.2.1 Timing Adjustable
No memory/cognitive test without an alternative	3.3.8 Accessible Authentication (Minimum)
Sufficient contrast; not colour-only feedback	1.4.3 Contrast (Minimum), 1.4.1 Use of Color
Text resizes to 200% / reflows	1.4.4 Resize Text, 1.4.10 Reflow
Correct accessible name/role/value on every control	4.1.2 Name, Role, Value
Visible label text matches the accessible name	2.5.3 Label in Name

4. Design Artifacts

I used five diagrams to work through this design. Each one is a standard technique for its purpose, not something I made up for this project.

Figure 1 — user journey map

I started here because a journey map is a common way to lay out what a user goes through step by step [12]. This one follows a screen-reader user through a full login across all three factors, showing what gets announced (spoken/ `aria-live`) at each step, plus the lockout/recovery path that can happen from any of the three steps.

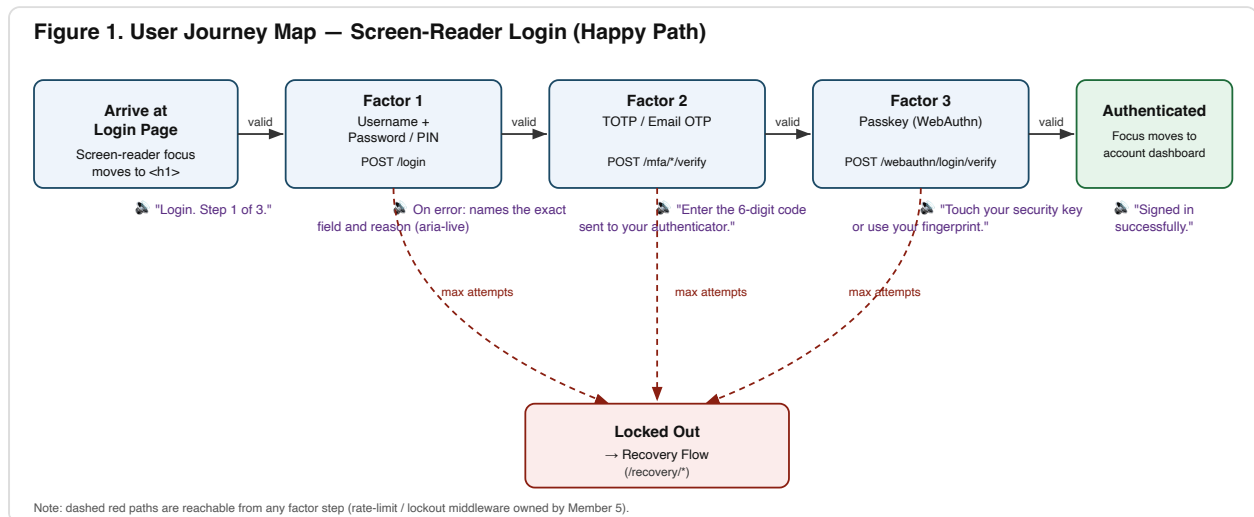


Figure 1. User journey map for a screen-reader login, annotated with spoken/aria-live announcements at each stage and the lockout/recovery branch.

Figure 2 — annotated wireframe

A simple wireframe of the first login screen, numbered in tab order, with a side panel explaining the ARIA role, accessible name, and relevant WCAG rule for each control. This is a normal way to check a screen's accessibility before anyone writes code for it.

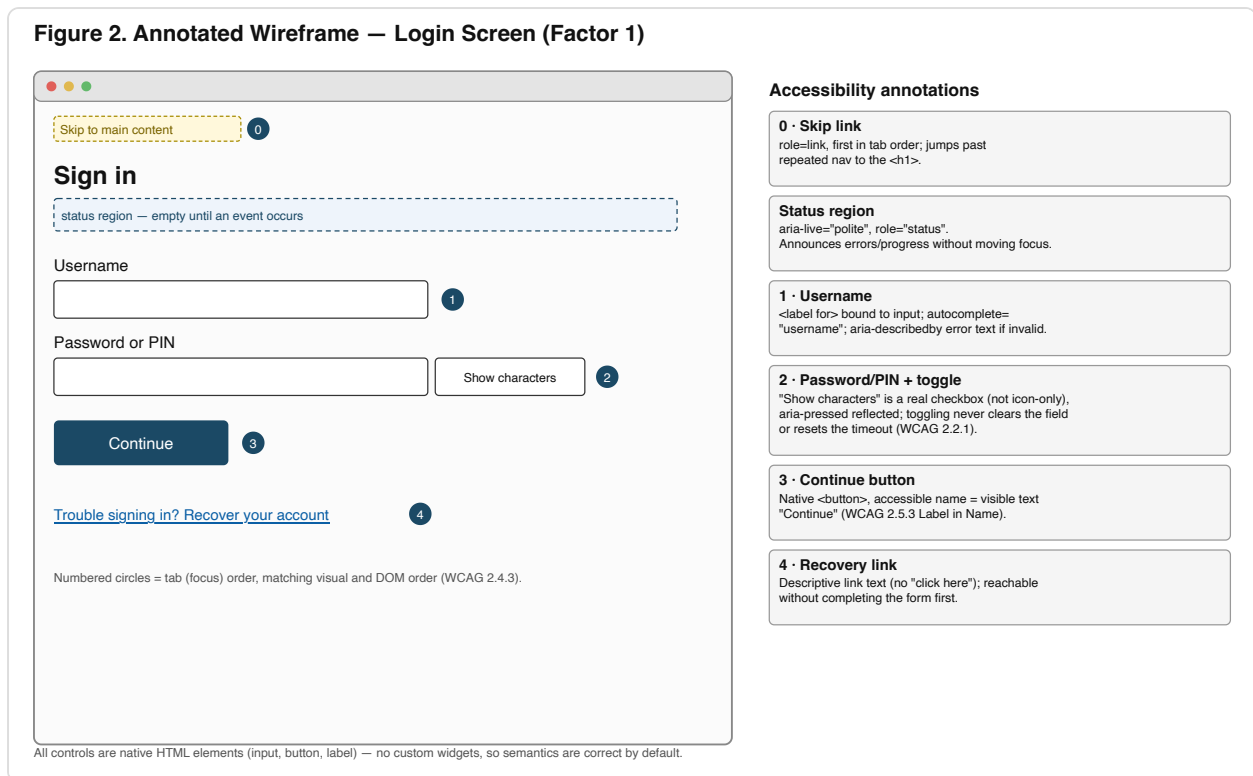


Figure 2. Annotated wireframe of the login screen, numbered by tab order with ARIA/WCAG callouts.

Figure 3 — sequence diagram

A sequence diagram with four lanes — User, Assistive Technology, Client, Server — walking through the OTP step. It shows exactly where focus moves, where the live-region announcement happens, and where the API call from `docs/api-contract.md` happens, relative to each other. Getting that order wrong is where a lot of accessibility bugs actually happen.

Figure 3. Sequence Diagram — Completing the Factor 2 (OTP) Step

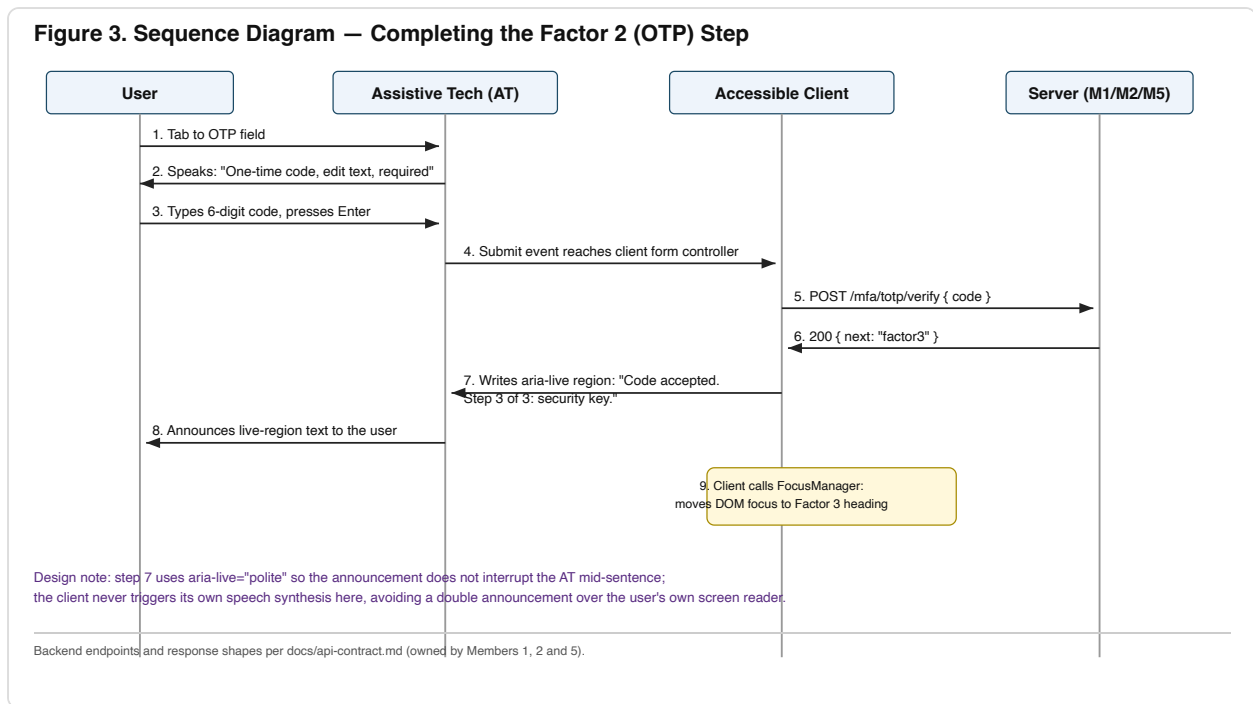


Figure 3. Sequence diagram of the Factor-2 (OTP) verification step across User, Assistive Technology, Client, and Server.

Figure 4 — client state diagram

The client's own view of the login process: **Unauthenticated** → **Factor1Pending** → **Factor2Pending** → **Factor3Pending** → **Authenticated**, with a **LockedOut** state that can be reached from any pending step, showing what gets announced when each state starts. This is a read-only copy of the real state kept on the server — the client never decides by itself that someone is logged in, which is part of how the system stops a user from skipping a step (threat T5 in the shared threat model).

Figure 4. Client-Side Authentication State Diagram

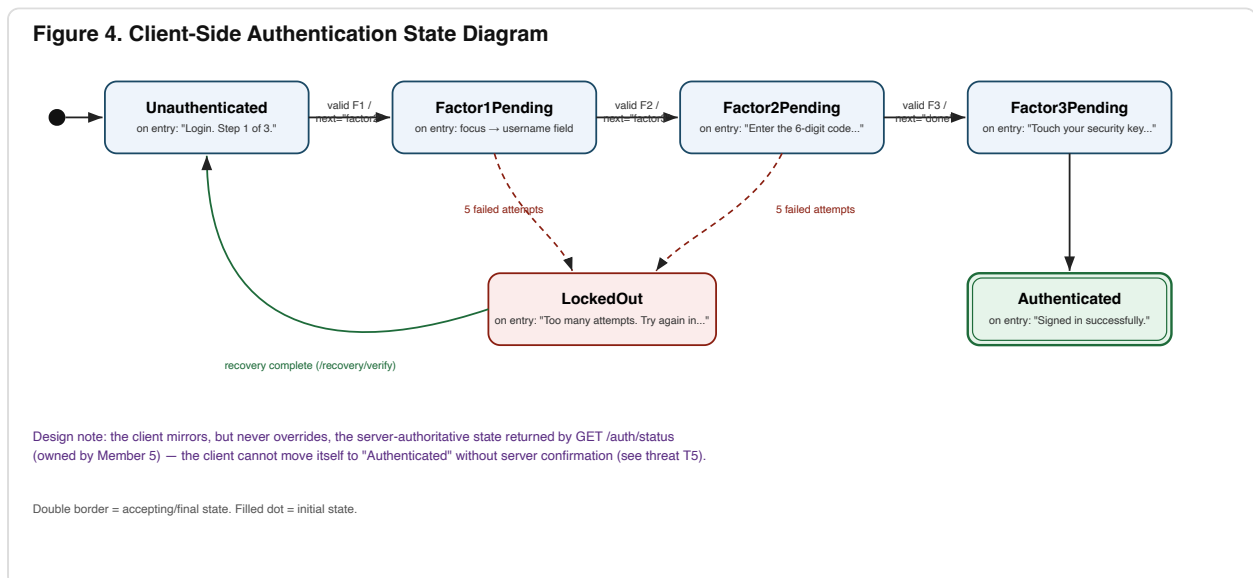


Figure 4. Client-side authentication state diagram, mirroring the server-authoritative state machine.

Figure 5 — component diagram

How the client code is actually organised: a main controller that coordinates four smaller pieces — one for moving focus, one for live-region announcements, one for optional speech, one for contrast/text size — plus one piece that's the only thing allowed to talk to the server. Splitting it up this way means I can test the accessibility behaviour on its own, without dragging login logic into every test.

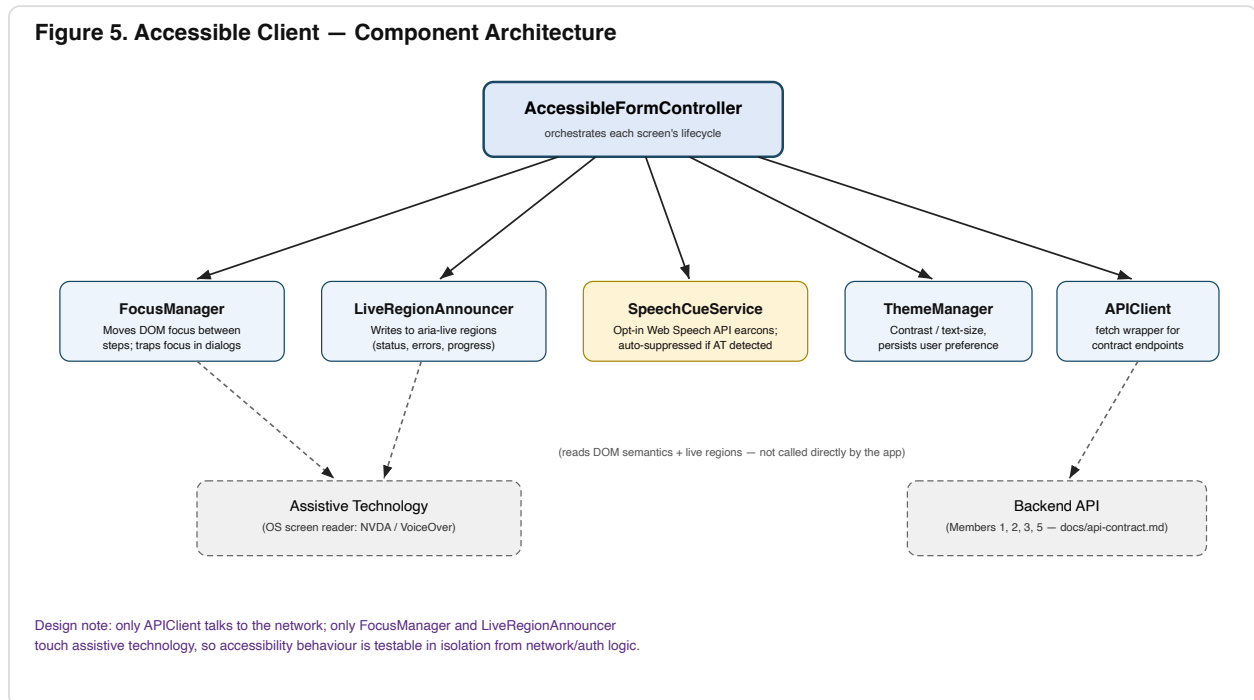


Figure 5. Component architecture of the accessible client.

5. Design Decisions

These are the decisions that came out of the requirements and diagrams above, along with the reasoning and evidence behind each one.

1. **The TOTP secret is always given as spoken/copyable text, never as a QR code only.** A QR code is a purely visual, camera-scanned thing with no built-in accessible option. WCAG's SC 3.3.8 specifically says authentication shouldn't rely on one sense with no alternative [4], so this wasn't really a choice.
2. **Status changes are announced through an ARIA live region** (`aria-live="polite", role="status"`) instead of a visual-only banner. This is the pattern the WAI-ARIA Authoring Practices Guide recommends for this exact situation [5], and it works because live regions are part of the WAI-ARIA 1.2 standard [6] that every major screen reader already supports.

3. **The Web Speech API is opt-in only, and turns itself off automatically if a screen reader is already running.** Two voices talking over the same event — the OS screen reader and the page's own speech — fight for attention and make things harder to follow, not easier. The ARIA guide's advice on live regions warns against exactly this kind of repeated announcement [5]. Since the API itself is still a draft rather than a finished standard [7], treating it as extra rather than something the app depends on felt like the safer choice.
 4. **Every timed step gets a generous, extendable timeout.** This is required by WCAG SC 2.2.1 [3]. Screen-reader navigation happens one item at a time rather than all at once the way sighted scanning does, so a short fixed timeout unfairly punishes exactly the users this system is for.
 5. **Everything can be operated by keyboard, in an order that matches the layout.** WCAG SC 2.1.1 and 2.4.3 [3]. Using plain HTML elements (`<button>` , `<input>` , `<label for>`) instead of custom widgets gets this right automatically, instead of me having to rebuild keyboard support by hand — this is shown in the wireframe in Figure 2.
 6. **A user-controlled high-contrast/large-text mode is available on every screen,** on top of respecting the browser's own contrast setting and zoom up to 400%. This covers WCAG SC 1.4.3, 1.4.4, and 1.4.10 [3], and it's really there for Persona B — someone who doesn't use a screen reader at all but still can't use a small, low-contrast layout.
 7. **Every control has a correct accessible name, role, and label** — using `<label for>` , `aria-describedby` for error text, and no icon-only buttons. WCAG SC 4.1.2 and 2.5.3 [3].
 8. **Error messages say exactly which field is wrong and why, in plain language, through the live region** — not colour alone. This covers WCAG SC 1.4.1 and 3.3.1 [3], and it also matches what the rest of the group agreed on: every error message from the back end has to make sense when read aloud with no visual context.
 9. **The client only trusts the server's own record of login progress — it never decides by itself that someone is "authenticated."** This is the client's part of stopping threat T5 in the shared threat model (a user trying to skip a step by calling a protected endpoint directly), and it's really a security decision as much as an accessibility one.
 10. **No visual CAPTCHA anywhere, including recovery.** A CAPTCHA that only works by sight would bring back exactly the problem SC 3.3.8 is meant to remove [4]. Abuse is handled instead by the shared rate-limiting and lockout system, which is the same kind of protection NIST SP 800-63B recommends for this [10].
-

6. Where Accessibility and Security Disagreed

Since this is a security module, it's worth being upfront about the places where accessible design and security didn't automatically agree, and how I settled each one.

Reading an OTP code out loud is riskier than a quick glance at the screen — someone nearby could overhear it (this is threat T7 in the shared threat model). I can't remove that risk completely, so instead I made speaking the code an explicit, separate action rather than something automatic, and the client suggests using headphones first. The code is always available silently too, as text the user's own screen reader can read, so audio is never the only way to get it.

Longer timeouts (decision 4 above) only apply to how long the user has to act — they don't extend how long an OTP or security-key request stays valid on the server, which other members control. So being patient with the user doesn't weaken the server's protection against replay attacks (threat T11).

And on decision 9 — it would have been simpler to let the client decide on its own when all three factors were done. I didn't do that on purpose: an inaccessible-but-secure screen is a bug I can still go back and fix, but a screen that's fast because it trusts itself instead of the server is a security hole. Given that choice, security won.

7. How I'd Test This

Here's how I'm testing my part, the same way the rest of the group is testing theirs:

1. Go through every screen in §3.2 from start to finish using NVDA, with the monitor off.
 2. Do the same using only Tab / Shift+Tab / Enter / Space — no mouse at all.
 3. Run an axe or Lighthouse scan on every screen and fix anything flagged as critical or serious.
 4. Manually check every rule in the §3.3 table against the WCAG 2.2 AA checklist.
 5. Turn off the Web Speech API completely, as if the browser didn't support it, and check nothing important is lost — only the optional sound cues should disappear. This is really the test for decision 3 above.
-

8. Limitations

- Deaf-blind users are out of scope for this version — Braille-display support wasn't something I could build or test in the time I had. That's a real limitation on who this system currently works for.
 - JAWS was treated as secondary because it's paid software, which is a budget limit on a student project, not a judgement that it matters less to real users.
 - I didn't get to test this with actual blind or low-vision users — there wasn't time or approval for that within the module. The design relies on WCAG 2.2, WAI-ARIA, and published usage data (the WebAIM survey) instead, which is the best evidence I had available, but it isn't a replacement for real usability testing.
-

9. References

- [1] World Health Organization, "Blindness and vision impairment," Fact Sheet, Feb. 2026. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>
- [2] WebAIM, "Screen Reader User Survey #10 Results," 2024. [Online]. Available: <https://webaim.org/projects/screenreadersurvey10/>
- [3] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, 2023. [Online]. Available: <https://www.w3.org/TR/WCAG22/>
- [4] W3C WAI, "Understanding SC 3.3.8: Accessible Authentication (Minimum)," WCAG 2.2. [Online]. Available: <https://www.w3.org/WAI/WCAG22/Understanding/accessible-authentication-minimum.html>
- [5] W3C WAI, "ARIA Authoring Practices Guide (APG)." [Online]. Available: <https://www.w3.org/WAI/ARIA/apg/>
- [6] W3C, "Accessible Rich Internet Applications (WAI-ARIA) 1.2," W3C Recommendation. [Online]. Available: <https://www.w3.org/TR/wai-aria-1.2/>
- [7] W3C Community Group, "Web Speech API," Draft Community Group Report. [Online]. Available: <https://wicg.github.io/speech-api/>
- [8] W3C, "Web Authentication: An API for accessing Public Key Credentials Level 2 (WebAuthn)," W3C Recommendation. [Online]. Available: <https://www.w3.org/TR/webauthn-2/>
- [9] M. Nystrom and J. Josefsson, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6238>
- [10] NIST, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B. [Online]. Available: <https://pages.nist.gov/800-63-4/sp800-63b.html>
- [11] OWASP Foundation, "Authentication Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[12] K. Kaplan, "Journey Mapping 101," Nielsen Norman Group, 2016. [Online]. Available: <https://www.nngroup.com/articles/journey-mapping-101/>

[13] W. W. Gaver, "Auditory icons: Using sound in computer interfaces," Human-Computer Interaction, vol. 2, no. 2, pp. 167–177, 1986.

[14] NV Access, "NVDA Screen Reader." [Online]. Available: <https://www.nvaccess.org/>